

MESSAGEFOUNDRY DOCUMENTATION

HL7 validation

MessageFoundry validates HL7 v2.x at three distinct tiers. Each catches a different class of problem; together they let a feed be both tolerant (real-world HL7 is frequently non-conformant) and safe (a safety-critical feed can reject anything off-spec).

MessageFoundry validates HL7 v2.x at three distinct tiers. Each catches a different class of problem; together they let a feed be both *tolerant* (real-world HL7 is frequently non-conformant) and *safe* (a safety-critical feed can reject anything off-spec). Pick the tiers a given feed needs — they compose.

Tier	Engine	When	Checks	On failure
1. Tolerant peek	<code>python-hl7</code> (<code>parsing/peek.py</code>)	Always (hot path)	Parse + fast field access; size/segment caps; MSH present	Unparseable / oversized → <code>ERROR</code> (NAK), never crashes the connection
2. Strict structural	<code>hl7apy</code> (<code>parsing/validate.py</code>)	Opt-in per inbound (<code>strict=True</code>)	Version-aware schema : segment cardinality, datatypes, table values, lengths, MSH-12 version	Non-conformant → synchronous NAK (AR/AE) at the listener
3. Business consistency	<code>parsing/consistency.py</code> (this WP)	In a Router/Handler	Cross-field coherence the schema can't express	Handler decides: <code>FILTERED</code> or <code>ERROR</code> /dead-letter

Tier 1 — tolerant peek (always on)

The hot path uses `python-hl7` for fast, forgiving field access (`msg["PID-3"]`), so routing never pays for full structural validation. It enforces only hard safety limits (max message bytes, max segments, MSH presence). A message that can't be parsed, or that exceeds a limit, is routed to the error/dead-letter path and logged `ERROR` — it never crashes the connection. This is the right default for most feeds: you accept what arrives, preserve the raw, and route bad messages to where an operator sees them.

Tier 2 — strict structural validation (opt-in)

For a feed where an off-spec message must be *rejected at the door*, enable `hl7apy` strict validation on the **inbound** connection. It is version-aware (checks the message against the official HL7 structure for its version) and is the slow path, so it is kept off routing and is **opt-in per connection**:

```
from messagefoundry import MLLP, inbound

inbound("IB_ACME_ADT", MLLP(port=2575), router="adt_router", strict=True, hl7_version="2.5")
```

A non-conformant message is **NAK'd synchronously** (AR/AE) at the listener, before it is ever routed — the sender is told immediately. Be **explicit about `hl7_version`** for a strict feed; don't rely on silent autodetection. Enable strict when: the downstream system is intolerant of malformed structure, the feed is contractually conformant, or correctness outweighs throughput. Leave it off for a tolerant archival/forwarding feed.

Strict validation surfaces a single conformance error (not a full report) — a full report of a PHI message would be a data leak. See [parsing/validate.py](#).

| Tier 3 — cross-field business consistency (Router/Handler)

Strict validation checks each item against the schema **independently**. It does **not** check that *combinations of related items are reasonable*: that a required identifier is present, that a value is echoed consistently across segments, or that admit ≤ discharge. Per **ASVS 2.2.3 / 2.1.2**, that combined-item consistency is the **application's** job — in a code-first engine, the **Router/Handler**.

`messagefoundry/parsing/consistency.py` provides small, **generic, composable** primitives. Each takes the parsed `Message` plus field *paths* and returns a list of **PHI-safe** `Violation`s (rule + path, **never the field value**):

Primitive	Checks
<code>required(msg, *paths)</code>	each path is present (non-empty)
<code>same_across(msg, *paths)</code>	the paths all hold the same value (catches differing values <i>and</i> present-vs-absent)
<code>valid_date(msg, path)</code>	the value is a well-formed HL7 date/time (when present)
<code>dates_in_order(msg, earlier, later)</code>	<code>earlier ≤ later</code> (when both present & valid)
<code>matches(msg, path, pattern)</code>	the value fully matches a regex (a field-format allow-list)
<code>check(*groups)</code>	flattens several results into one list for a single decision

The library **detects; the Handler decides**. Keeping it pure preserves the at-least-once *re-run* invariant (a Handler must be a pure function of the message — CLAUDE.md §2). Compose the generic primitives into your feed's message-type-specific rules:

```

from messagefoundry import Send, handler
from messagefoundry.parsing.consistency import (
    ConsistencyError, check, dates_in_order, required, same_across, valid_date,
)

@handler("validate_and_archive")
def validate_and_archive(msg):
    violations = check(
        required(msg, "PID-3", "PID-5", "MSH-10"),    # patient id, name, control id present
        same_across(msg, "MSH-9.2", "EVN-1"),         # trigger event echoed in EVN-1
        valid_date(msg, "PID-7"),                     # date of birth well-formed
        dates_in_order(msg, "PV1-44", "PV1-45"),      # admit ≤ discharge
    )
    if violations:
        raise ConsistencyError(violations)             # → ERROR / dead-letter (operator sees it)
        # ...or `return None` to drop it silently (logged FILTERED)
    return Send("FILE-OUT_ACME_ADT", msg)

```

Acting on a non-empty result is a deliberate choice:

- `raise ConsistencyError(violations)` → the message goes to the error/dead-letter path (logged `ERROR`, surfaced via the AlertSink). Use this when a downstream system can't safely consume it. The exception's string is built from rule + paths only, so it is **safe to log and store** (no PHI).
- `return None` → the message is dropped and logged `FILTERED`. Use this for a benign coherence issue you'd rather not forward.

A full worked example — wired inbound → router → handler → file — is in

`samples/consistency/validated_adt.py`:

```
python -m messagefoundry serve --config samples/consistency --db ./mf.db --env dev
```

PHI-safety

A `Violation` records the **rule and the field path(s)**, never the field *value*. So violations can be logged, aggregated, or ridden into a stored disposition (via the `safe_exc()` chokepoint — see [PHI.md](#) §7) without leaking PHI: a date violation reads *"PID-7 is not a valid HL7 date/time"*, never the offending value.

Choosing tiers

- **Most feeds:** Tier 1 only — tolerant, route bad messages to `ERROR`.
- **Add Tier 2** (`strict=True`) when off-spec *structure* must be rejected at the door.
- **Add Tier 3** when *business rules across fields* matter (required identifiers, sane/ordered dates, cross-segment coherence) — independent of whether Tier 2 is on.